

คุยกับ Local Server ของ LM Studio ด้วย REST API

พื้นฐาน HTTP, JSON และ REST จนถึงการใช้ Postman คุยกับ LM Studio ได้จริง — ครอบคลุมทั้งหมดไม่มี
API key และมี API key พร้อมส่วนขยายเรื่อง MCP

HTTP · JSON · REST Postman · LM Studio 0.4.16 Model Context Protocol

เนื้อหาทั้งหมดในสไลด์ชุดนี้

ไล่ตั้งแต่พื้นฐานไปจนถึงลงมือทำจริง 6 Lab และ MCP

00 เป้าหมายและสิ่งที่ต้องเตรียม

01 Client–Server คืออะไร

02 HTTP ฉบับเข้าใจใน 10 นาที

03 JSON ภาษาที่ใช้คุยกัน

04 REST API คืออะไร

05 เปิดเซิร์ฟเวอร์ใน LM Studio

06 รู้จัก Postman

07 Lab 1: GET รายชื่อโมเดล

08 Lab 2: POST คุยกับโมเดล

09 Lab 3: โหมด OpenAI-compatible

10 Lab 4: จำบทสนทนา + สตรีม

11 Lab 5: โหลด/ปลดโมเดล

12 มี key กับ ไม่มี key

13 MCP คืออะไร

14 Lab 6: ต่อ MCP ผ่าน API

15 แก้ปัญหาที่เจอบ่อย

16 แบบฝึกหัด

17 สรุป Cheat Sheet

18 ศัพท์ไทย–อังกฤษ + อ้างอิง

—

00 / 18

เป้าหมายและสิ่งที่ต้องเตรียม

- > รู้ว่าจบบทเรียนนี้แล้วจะทำได้บ้าง
- > เตรียมโปรแกรมและโมเดลให้พร้อมก่อนลงมือ

จบบทเรียนนี้แล้วคุณจะทำอะไรได้

- อธิบายได้ว่า HTTP request หนึ่งก้อนประกอบด้วยอะไรบ้าง (**method**, **URL**, **headers**, **body**)
- อ่านและเขียน **JSON** ได้ถูกไวยากรณ์ (syntax)
- เปิด local server ของ LM Studio และรู้ว่าสวิตช์แต่ละตัวในหน้า Server Settings ทำอะไร
- ใช้ **Postman** ยิง `GET` และ `POST` ไปที่ `http://localhost:1234` แล้วอ่านผลลัพธ์เป็น
- ตั้งค่าและใช้ **API key** (Bearer token) รวมถึงแก้ error `401 Unauthorized` ได้
- เข้าใจว่า **MCP** (Model Context Protocol) คืออะไร และเรียกใช้เครื่องมือภายนอกผ่าน API ของ LM Studio ได้

สิ่งที่ต้องเตรียมก่อนเริ่ม

- 1 **LM Studio** เวอร์ชัน 0.4.x ขึ้นไป (ตัวอย่างในสไลด์นี้ใช้ 0.4.16) ดาวน์โหลดที่ lmstudio.ai
- 2 **โมเดลอย่างน้อย 1 ตัว** ที่ดาวน์โหลดไว้แล้ว เช่น `ibm/granite-4-micro` ตัวเล็ก เร็ว เหมาะกับการทดลอง
- 3 **Postman** แบบติดตั้งลงเครื่อง (desktop app) ดาวน์โหลดที่ postman.com/downloads

ทำไมต้องเป็น Postman แบบติดตั้งลงเครื่อง

Postman เวอร์ชันเว็บรันอยู่ในเบราว์เซอร์บนคลาวด์ จึงมองไม่เห็น `localhost` ของเครื่องคุณ ต้องติดตั้งโปรแกรมเสริม Postman Agent เพิ่ม — ใช้แบบ desktop app ง่ายกว่ามาก

01 / 18

Client–Server คืออะไร

- สองฝ่ายที่แบ่งหน้าที่ชัดเจน: ผู้รอ กับ ผู้ขอ
- คำศัพท์พื้นฐาน: localhost, port, LAN IP

โปรแกรมสองฝั่งคุยกันโดยแบ่งหน้าที่ชัดเจน

เซิร์ฟเวอร์ (server)

ฝ่ายที่ “นั่งรอ” ให้คนมาขอของ ไม่เริ่มคุยก่อน — ในกรณีของเรา คือ LM Studio ที่เปิดโหมด Developer ไว้

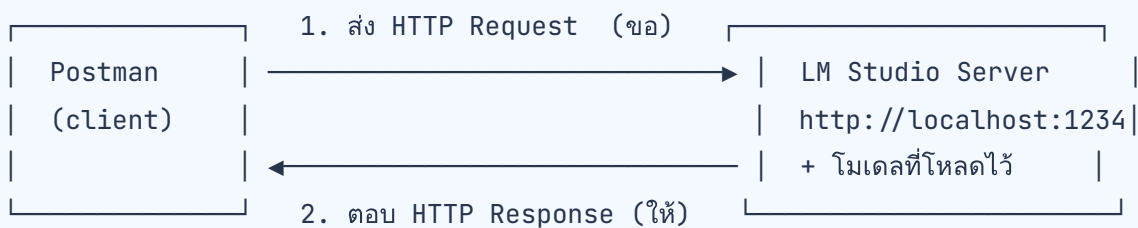
ไคลเอนต์ (client)

ฝ่ายที่ “เดินไปขอ” — ในกรณีของเรา คือ Postman (หรือโค้ด Python/เว็บแอปที่คุณจะเขียนต่อไป)

API (Application Programming Interface) แปลตรงตัวคือ “หน้าต่างที่โปรแกรมใช้คุยกับโปรแกรม”

เปรียบเทียบ: หน้าจอ LM Studio คือหน้าร้านสำหรับ *คน* ส่วน API คือช่องสั่งของสำหรับ *โปรแกรม* — ทั้งสองทางเข้าถึงโมเดลตัวเดียวกัน

เส้นทางของ Request และ Response



Postman เป็นฝ่าย **เริ่มคุยก่อนเสมอ** ส่ง request ไป แล้วรอ LM Studio ตอบกลับมาเป็น response — วนซ้ำแบบนี้ทุกครั้งที่คุณกด Send

คำสำคัญ 3 คำที่มักสับสน

คำ	ความหมาย	ตัวอย่างในบทเรียนนี้
localhost	ชื่อเรียกเครื่องของตัวเอง เท่ากับหมายเลข 127.0.0.1 เสมอ	<code>http://localhost:1234</code>
พอร์ต (port)	เลขช่องประตูของเครื่อง โปรแกรมแต่ละตัวของคนละช่อง	LM Studio ใช้ 1234 โดยค่าเริ่มต้น
IP ในวง LAN	ที่อยู่ของเครื่องคุณในเครือข่ายบ้าน/ออฟฟิศ ให้เครื่องอื่นเรียกเข้ามาได้	<code>http://192.168.56.1:1234</code>

localhost กับ 192.168.x.x ต่างกันยังไง

`localhost` ใช้ได้เฉพาะจากเครื่องเดียวกัน ส่วน `192.168.x.x` คือที่อยู่เพื่อนในวงเน็ตเดียวกันเรียกเข้ามาได้ — ต้องเปิด **Serve on Local Network** ก่อน (หัวข้อ 05)

02 / 18

HTTP ฉบับเข้าใจใน 10 นาที

- › โครงสร้าง request / response
- › Method, Status code, Header ที่ต้องรู้จัก

HTTP คือ “ภาษากลาง” ระหว่าง client และ server

HTTP (HyperText Transfer Protocol) คือรูปแบบมาตรฐานที่ client กับ server ใช้คุยกัน ทุกครั้งที่คุณเปิดเว็บ กดไลก์ หรือยิง API เบื้องหลังคือ HTTP ทั้งหมด

4

ส่วนประกอบของ request: method, path, headers, body

3

หลักของ response: status code, headers, body

1234

พอร์ตเริ่มต้นของ LM Studio

หน้าตาของ Request (ฝั่งขอ)

● HTTP REQUEST

```
POST /api/v1/chat HTTP/1.1      ← method + path
Host: localhost:1234            ← headers (ข้อมูลกำกับ)
Content-Type: application/json
Authorization: Bearer lms-xxxxx

{"model": "ibm/granite-4-micro",  ← body (เนื้อหาที่ส่งไป)
 "input": "สวัสดี"}
```

- 1 **Method** — กริยา บอกว่าจะ “ทำอะไร” เช่น `GET` = ขอ, `POST` = ส่งของไปให้
- 2 **URL / Path** — คำนาม บอกว่าจะทำ “กับอะไร” เช่น `/api/v1/models`
- 3 **Headers** — ข้อมูลกำกับพัสดุ เช่น เนื้อหาเป็นชนิดไหน มีบัตรผ่านไหม
- 4 **Body** — เนื้อหาจริงที่ส่งไป `GET` มักไม่มี body ส่วน `POST` มี

แกะ URL ที่ละชิ้น

```
http://localhost:1234/api/v1/chat?limit=10
```

```
scheme  host  port  path  query string
```

(โพรโทคอล) (เครื่อง) (พอร์ต) (เส้นทาง) (พารามิเตอร์เสริม)

Method ที่ต้องรู้จัก

Method	ใช้ทำอะไร	มี body ไหม	คุณสมบัติ
GET	ขอข้อมูล ไม่เปลี่ยนแปลงอะไรบนเซิร์ฟเวอร์	ไม่มี	safe + idempotent
POST	ส่งข้อมูลไปให้ประมวลผล / สร้างของใหม่	มี	ไม่ safe, ไม่ idempotent
PUT	แทนที่ข้อมูลเดิมทั้งก้อน	มี	idempotent
PATCH	แก้บางส่วนของข้อมูลเดิม	มี	ไม่การันตี idempotent
DELETE	ลบข้อมูล	ปกติไม่มี	idempotent

idempotent แปลว่าอะไร

“ทำกี่ครั้งผลลัพธ์เท่าเดิม” — ลบไฟล์เดิมซ้ำ 5 ครั้ง ไฟล์ก็หายไป 1 ไฟล์ แต่ POST ส่งชื่อของ 5 ครั้ง = ได้ของ 5 ชิ้น สำคัญมากเวลาต้อง retry (ยิงซ้ำเมื่อเน็ตหลุด)

หน้าตาของ Response (ฝั่งตอบ)

● HTTP RESPONSE

```
HTTP/1.1 200 OK           ← status code + ข้อความ
Content-Type: application/json ← headers

{"models": [ ... ]}      ← body (ผลลัพธ์)
```

Response ประกอบด้วย **status code** (บอกผลลัพธ์), **headers** (ข้อมูลกำกับ เช่น Content-Type) และ **body** (เนื้อหาจริงหรือรายละเอียด error)

Status code ที่ต้องอ่านออก

กลุ่ม	รหัส	ความหมาย	ในบริบท LM Studio
2xx สำเร็จ	200 OK	เรียบร้อย	ได้คำตอบจากโมเดลแล้ว
4xx ผิดที่ฝั่งเรา	400	ส่งข้อมูลผิดรูป	JSON พิมพ์ผิด หรือลืมพาด์ model
	401	ไม่มีบัตรผ่าน / บัตรผิด	เปิด Require Authentication แต่ไม่ได้ใส่ token
	403	มีบัตรแต่สิทธิไม่พอ	token นี้ไม่ได้รับ permission ที่จำเป็น
	404	ไม่มีเส้นทางนี้	พิมพ์ path ผิด เช่น /api/v1/chats
5xx ผิดที่เซิร์ฟเวอร์	500	เซิร์ฟเวอร์พัง	โมเดลโหลดไม่ขึ้น / RAM ไม่พอ

จำง่าย ๆ
4xx = “คุณพิมพ์อะไรผิดสักอย่าง” · 5xx = “ฝั่งเซิร์ฟเวอร์มีปัญหา” · ไม่ได้รหัสอะไรเลยแต่ขึ้น *ECONNREFUSED* แปลว่าเซิร์ฟเวอร์ยังไม่เปิดหรือพอร์ตผิด

Header ที่ใช้บ่อยที่สุด 2 ตัว

Header	ค่า	แปลว่า
Content-Type	application/json	“เนื้อหาที่ผมส่งไปเป็น JSON นะ ช่วยอ่านแบบ JSON ด้วย”
Authorization	Bearer <token>	“นี่คือบัตรผ่านของผม” (ใช้เมื่อเปิดระบบ key เท่านั้น)

03 / 18

JSON — ภาษาที่ใช้คุยกัน

- › ชนิดข้อมูล 6 แบบ
- › กฎเหล็กที่มือใหม่พลาดบ่อย

JSON คืออะไร และมีชนิดข้อมูลแค่ 6 แบบ

JSON (JavaScript Object Notation) คือรูปแบบข้อความสำหรับเก็บข้อมูลที่ทั้งคนและเครื่องอ่านได้ เกือบทุก API สมัยใหม่รับ-ส่งด้วย JSON รวมถึง LM Studio

● JSON

```
{  
  "text":    "ข้อความต้องอยู่ในเครื่องหมายคำพูด",  
  "number":  0.7,  
  "boolean": true,  
  "nothing": null,  
  "array":   ["a", "b", "c"],  
  "object":  { "key": "value" }  
}
```

กฎเหล็กที่มือใหม่พลาดบ่อย

✗ ผิด

```
{
  'model': 'granite',    ← ใช้ ' เดี่ยว
  input: "hi",           ← ไม่มี "
  "temp": 0.7,           ← ลูกรั่วเกินท้าย
}
```

✓ ถูก

```
{
  "model": "granite",
  "input": "hi",
  "temperature": 0.7
}
```

- ต้องใช้เครื่องหมายคำพูดคู่ (double quote) `"` เท่านั้น
- ชื่อคีย์ (key) ต้องมีเครื่องหมายคำพูดครอบเสมอ · ห้ามมีลูกน้ำตัวสุดท้ายก่อนปิดวงเล็บ
- JSON ไม่มีคอมเมนต์ ใส่ `//` ไม่ได้ · ภาษาไทยใช้ได้ปกติเพราะเป็น UTF-8

เช็ก JSON ก่อนยิงทุกครั้ง

ใช้ปุ่ม Beautify ใน Postman

เมื่อเลือก Body → raw → JSON แล้ว ถ้า JSON ผิดจะมีขีดเส้นใต้สีแดงเตือนทันที กดปุ่ม **Beautify** (มุมขวาของช่อง body) เพื่อจัดรูปให้อ่านง่ายและตรวจสอบวงเล็บได้ง่ายขึ้น

04 / 18

REST API คืออะไร

- › ทุกอย่างคือ "ทรัพยากร (resource)"
- › ข้อกำหนดหลัก 6 ข้อของ REST

ทุกอย่างที่ตั้งชื่อได้ คือ “ทรัพยากร”

REST (REpresentational State Transfer) ไม่ใช่โปรแกรมหรือไลบรารี แต่เป็น *สไตล์การออกแบบ* ที่ Roy Fielding เสนอไว้ API ที่ทำตามสไตล์นี้เรียกว่า **RESTful API**

- `/api/v1/models` → ทรัพยากร “รายชื่อโมเดลทั้งหมด”
- `/api/v1/chat` → ทรัพยากร “การสนทนา”

สังเกตว่า **path** เป็นคำนาม ส่วนกริยาอยู่ที่ **method** — จึงไม่เขียน `/getModel`s แต่เขียน `GET /models` แทน

ข้อกำหนดหลัก 6 ข้อของ REST

ข้อกำหนด	ความหมายแบบภาษาคน
Client–Server	แยกหน้าที่ระหว่างฝ่ายแสดงผลกับฝ่ายเก็บข้อมูล ต่างฝ่ายต่างพัฒนาได้อิสระ
Stateless (ไร้สถานะ)	ทุก request ต้องมีข้อมูลครบในตัวเอง เซิร์ฟเวอร์ไม่จำว่าคุณเคยส่งอะไรมาก่อน
Cacheable	คำตอบต้องบอกได้ว่าเก็บไว้ใช้ซ้ำได้หรือไม่ เพื่อลดภาระเซิร์ฟเวอร์
Uniform Interface	ใช้รูปแบบมาตรฐานเดียวกัน: ระบุทรัพยากรด้วย URI, ส่งเป็น representation, ข้อความอธิบายตัวเองได้
Layered System	ระหว่างทางอาจมี proxy/gateway คั่น client ไม่จำเป็นต้องรู้
Code on Demand (ไม่บังคับ)	เซิร์ฟเวอร์ส่งโค้ดให้ client รับได้ เช่นส่ง JavaScript

representation คืออะไร — และข้อยกเว้นของ LM Studio

ตัวทรัพยากรจริง ๆ (เช่น โมเดล LLM ที่อยู่ใน RAM) ส่งผ่านสายไม่ได้ สิ่งที่ส่งไป-มาจึงเป็นภาพแทนของมันในรูปข้อความ ซึ่งในที่นี้คือ JSON — นี่คือการมาของตัว R และ S ในคำว่า REST

ข้อสังเกตสำคัญ: LM Studio เลือกทำผิดกฎ stateless โดยตั้งใจ

REST แท้บอกว่าเซิร์ฟเวอร์ต้องไม่จำอะไร แต่ endpoint `/api/v1/chat` เป็นแบบ **stateful** คือจำบทสนทนาให้ คุณจึงส่งต่อบทสนทนาเดิมด้วย `previous_response_id` ได้ (หัวข้อ 10) — ส่วน endpoint แบบ OpenAI-compatible ยังคง stateless ตามปกติ

05 / 18

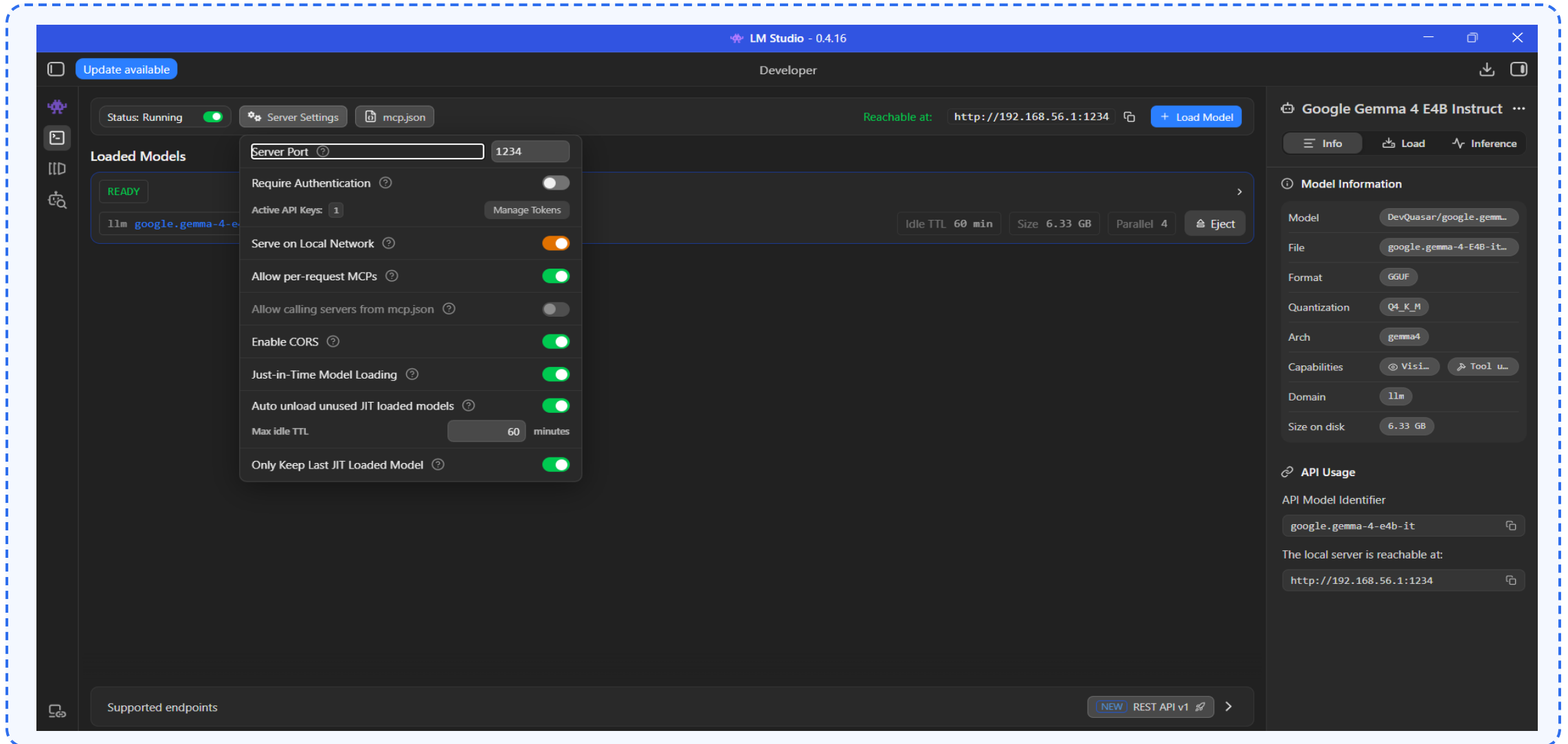
เปิดเซิร์ฟเวอร์ใน LM Studio

- › ขั้นตอนเปิด Developer server
- › อธิบายทุกสวิตช์ใน Server Settings
- › แผนที่ endpoint ทั้งหมด

เปิดเซิร์ฟเวอร์ที่ละขั้น

- 1 เปิด LM Studio แล้วมองแถบไอคอนด้านซ้าย เลือกไอคอน **Developer** (รูปเทอร์มินัล `>_`)
- 2 สลับสวิตช์ **Status** ให้เป็น **Running** (สีเขียว)
- 3 ดูข้อความ **Reachable at:** มุมขวบน — นั่นคือที่อยู่เซิร์ฟเวอร์ของคุณ เช่น `http://192.168.56.1:1234` จากเครื่องตัวเองใช้ `http://localhost:1234` ได้เหมือนกัน
- 4 กด **Server Settings** เพื่อดูสวิตช์ต่าง ๆ
- 5 ดูช่อง **Developer Logs** ด้านล่าง — ทุกครั้งที่ยิง request จะมี log ขึ้นที่นี่ ใช้ดีบั๊กได้ดีมาก

หน้าจอ Developer → Server Settings



อธิบายทุกสวิตช์ (1/2)

สวิตช์	ทำอะไร	แนะนำสำหรับห้องเรียน
Server Port	เลขพอร์ตที่เซิร์ฟเวอร์ใช้ ค่าเริ่มต้น 1234	ปล่อยไว้ 1234
Require Authentication	บังคับให้ทุก request ต้องแนบ API token ไม่งั้นตอบ 401	เริ่มด้วยปิด แล้วค่อยเปิดในหัวข้อ 12
Manage Tokens	ปุ่มเปิดหน้าจอสราง/ลบ/แก้ไขสิทธิ์ของ token	ใช้เมื่อเปิดข้อบน
Serve on Local Network	ยอมให้เครื่องอื่นในวง LAN เรียกเข้ามาได้	เปิดเฉพาะตอนต้องให้เพื่อนยังเข้ามา + เปิด Auth คู่กัน

อธิบายทุกสวิตช์ (2/2)

สวิตช์	ทำอะไร	แนะนำสำหรับห้องเรียน
Allow per-request MCPs	ยอมให้ request ระบุ MCP server ชั่วคราวมาในตัว request ได้	เปิด ถ้าจะทำ Lab 6
Allow calling servers from mcp.json	ยอมให้ request เรียก MCP server ที่ตั้งไว้ล่วงหน้า	เปิดเมื่อจำเป็น — ต้องใช้ API token เสมอ
Enable CORS	ยอมให้ JavaScript หน้าเว็บ (คนละ origin) เรียก API นี้ได้	เปิดถ้าจะทำเว็บแชตบอทเรียกเข้ามา
Just-in-Time Model Loading	ถ้ายังไม่โหลดโมเดล แต่ request ระบุชื่อมา จะโหลดให้อัตโนมัติ	เปิด — สะดวกมากสำหรับผู้เริ่มต้น
Auto unload + Max idle TTL	ปลดโมเดลออกจากหน่วยความจำเมื่อไม่มีใครใช้เกิน N นาที	เปิด เพื่อไม่ให้ RAM/VRAM เต็ม

คำเตือนเรื่องความปลอดภัย

เปิด Serve on Local Network โดยไม่เปิด Require Authentication = ใครในวงเน็ตเดียวกันยังคำสั่งใช้โมเดลบนเครื่องคุณได้ฟรี รวมถึงสั่งดาวนี่โหลดโมเดลลงเครื่องคุณได้ด้วย

แผนที่ endpoint ทั้งหมดที่ LM Studio เปิดให้

LM Studio มี API ให้เลือกใช้ 3 ชุด ซึ่งใช้พอร์ตเดียวกัน แตกกันที่ path

ชุด	ตัวอย่าง path	เหมาะกับ
Native REST API v1 ใหม่สุด	/api/v1/chat, /api/v1/models	จำลองสนทนา, ต่อ MCP, โหลด/ดาวนโหลดโมเดล
OpenAI-compatible	/v1/chat/completions, /v1/models	ย้ายโค้ดที่เขียนกับ OpenAI มารันบนเครื่องตัวเอง
REST API v0	/api/v0/chat/completions	เหมือน OpenAI แต่แถบข้อมูลละเอียดกว่า เช่น tokens/sec, TTFT

Native REST API v1 — รายการ endpoint สำคัญ

Method + Path	หน้าที่
GET /api/v1/models	ดูรายชื่อโมเดลทั้งหมด พร้อมสถานะว่าโหลดอยู่ไหม
POST /api/v1/chat	ส่งข้อความให้โมเดลตอบ (รองรับ MCP, รูปภาพ, สตริม)
POST /api/v1/models/load	สั่งโหลดโมเดลเข้าหน่วยความจำ
POST /api/v1/models/unload	สั่งปลดโมเดลออกจากหน่วยความจำ
POST /api/v1/models/download	สั่งดาวน์โหลดโมเดลใหม่ คืนค่า <code>job_id</code>
GET /api/v1/models/download/status/:job_id	เช็คความคืบหน้าการดาวน์โหลด

06 / 18

รู้จัก Postman

- ส่วนประกอบหน้าจอที่ต้องรู้จัก
- ตั้งค่า Environment ครั้งเดียว ใช้ได้ทั้งคอร์ส

ส่วนประกอบหน้าจอ Postman ที่ต้องรู้จัก

Postman คือโปรแกรมสำหรับ “ยิง” HTTP request ด้วยมือ โดยไม่ต้องเขียนโค้ด เห็นทุกส่วนของ request อย่างชัดเจน

ส่วน	อยู่ตรงไหน	ใช้ทำอะไร
Method dropdown	ซ้ายสุดของแถบ URL	เลือก GET / POST / ...
URL bar	กลางบน	พิมพ์ที่อยู่ปลายทาง
Authorization	แท็บใต้ URL	ใส่ token แบบมีตัวช่วย
Headers	แท็บใต้ URL	ใส่ <code>Content-Type</code> ฯลฯ
Body	แท็บใต้ URL	ใส่ JSON ที่จะส่งไป (raw → JSON)
Send / Response pane	ขวาบน / ครึ่งล่าง	ยิง (Ctrl+Enter) แล้วดู body, status, เวลา

ตั้งค่าครั้งเดียว ใช้ได้ทั้งคอร์ส: Environment

- 1 คลิกไอคอน **Environments** แถบซ้าย → Create Environment → ตั้งชื่อ `LM Studio`
- 2 เพิ่มตัวแปร: ชื่อ `baseUrl` ค่า `http://localhost:1234`
- 3 เพิ่มอีกตัว: ชื่อ `token` ค่าเว้นว่างไว้ก่อน — ตั้ง Type เป็น **secret**
- 4 กด Save แล้วเลือก environment นี้จาก dropdown มุมขวามือ
- 5 ต่อจากนี้พิมพ์ URL ได้เป็น `{{baseUrl}}/api/v1/models`

สร้าง Collection ไว้ด้วย

กด Collections → + ตั้งชื่อ `Week5 - LM Studio` แล้ว Save ทุก request ที่ทำใน Lab ไว้ในนั้น ย้อนกลับมาอ้างอิงซ้ำได้ และ Export ส่งให้เพื่อน/อาจารย์ได้

07-11 / 18

ลงมือจริง: Lab 1-5

- › GET รายชื่อโมเดล → POST ค่อยกับโมเดล
- › โมเดล OpenAI-compatible → จำบทสนทนา + สตรีม → จัดการโมเดล

GET รายชื่อโมเดล (ยังไม่มี key)

เป้าหมาย: ยิง request แรกในชีวิตให้สำเร็จ — ไม่มี body ไม่มี header ใด ๆ

- 1 ใน Postman กด + เปิดแท็บใหม่
- 2 เลือก method **GET**
- 3 พิมพ์ URL: `{{baseUrl}}/api/v1/models`
- 4 กด Send

CURL

```
curl http://localhost:1234/api/v1/models
```

ผลลัพธ์ที่ควรได้ (ย่อ)

● 200 OK — JSON RESPONSE

```
{
  "models": [
    {
      "type": "llm",
      "publisher": "google",
      "key": "google/gemma-4-26b-a4b",
      "display_name": "Gemma 4 26B A4B",
      "quantization": { "name": "Q4_K_M", "bits_per_weight": 4 },
      "loaded_instances": [],
      "max_context_length": 262144,
      "capabilities": { "vision": true, "trained_for_tool_use": true }
    }
  ]
}
```

สิ่งที่ต้องสังเกต

- ตัวเลข **200 OK** มุมขวามือของ response pane
- ฟیلด์ `key` = ชื่อโมเดลที่ต้องเอาไปใส่ในฟیلด์ `model` ของ Lab ถัดไป — **คัดลอกเก็บไว้**
- `loaded_instances` ว่าง `[]` = โมเดลยังไม่ถูกโหลดเข้าหน่วยความจำ (ไม่เป็นไร ถ้าเปิด JIT ไว้)
- ลองสลับไปดูช่อง Developer Logs ใน LM Studio จะเห็น log ของ request ที่เพิ่งยิงไป

ถ้าขึ้น **"Error: connect ECONNREFUSED 127.0.0.1:1234"**

แปลว่ายังถูกที่แล้วแต่ไม่มีใครรับสาย → กลับไปเช็คว่ Status ใน LM Studio เป็น Running และเลขพอร์ตตรงกัน

POST กับโมเดล

เป้าหมาย: ส่ง JSON body ก้อนแรก แล้วให้โมเดลตอบกลับ

- 1 แท็บใหม่ → method **POST**
- 2 URL: `{{baseUrl}}/api/v1/chat`
- 3 แท็บ Body → raw → เปลี่ยนเป็น **JSON**
- 4 วาง JSON โดยแก้ `model` ให้ตรงกับ `key` จาก Lab 1
- 5 กด Send แล้วรอ (ครั้งแรกอาจช้าเพราะ JIT กำลังโหลดโมเดล)

JSON BODY

```
{
  "model": "ibm/granite-4-micro",
  "input": "อธิบายว่า REST API คืออะไร ในสามประโยค"
}
```

ผลลัพธ์ที่ได้ และวิธีอ่าน

● 200 OK

```
{
  "model_instance_id": "ibm/granite-4-micro",
  "output": [
    { "type": "message",
      "content": "REST API คือ..." }
  ],
  "stats": {
    "input_tokens": 17,
    "total_output_tokens": 50,
    "tokens_per_second": 51.03,
    "time_to_first_token_seconds": 0.814
  },
  "response_id": "resp_0182bd7c..."
}
```

ฟิลด์

ความหมาย

output

อาร์เรย์ของสิ่งที่โมเดลผลิต คำตอบอยู่ที่ `output[0].content`

response_id

รหัสคำตอบนี้ ใช้คุยต่อได้ (หัวข้อ 10)

tokens_per_second

ความเร็วผลิตคำตอบ

time_to_first_token

TTFT — รอที่วินาทีที่ตัวอักษรแรกจะออก

ใส่ตัวเลือกเพิ่ม (optional parameters)

● JSON BODY – ตัวเต็ม

```
{
  "model": "ibm/granite-4-micro",
  "system_prompt": "คุณเป็นติวเตอร์วิชาเครือข่าย ตอบสั้น กระชับ",
  "input": "HTTP กับ HTTPS ต่างกันอย่างไร",
  "temperature": 0.7,
  "max_output_tokens": 300,
  "context_length": 8000,
  "reasoning": "off",
  "stream": false,
  "store": true
}
```

พารามิเตอร์ทั้งหมดของ /api/v1/chat

พารามิเตอร์	ชนิด	ความหมาย
model (บังคับ)	string	ชื่อโมเดลที่จะใช้
input (บังคับ)	string/array	ข้อความ หรือ array ใส่รูปภาพ base64 ได้
system_prompt	string	คำสั่งกำกับบุคลิก/กติกา
temperature	0-1	ความสร้างสรรค์ ยิ่งต่ำยิ่งนิ่ง
max_output_tokens	number	จำกัดความยาวคำตอบ
context_length	number	ขนาดหน้าต่างความจำ
reasoning	string	off/low/medium/high/on
stream	boolean	true = ทอยส่งคำตอบแบบ SSE
store	boolean	true (ค่าเริ่มต้น) = เก็บบทสนทนาไว้คุยต่อ
previous_response_id	string	resp_... ของรอบก่อน เพื่อคุยต่อเนื่อง
integrations	array	เปิดใช้ MCP server (หัวข้อ 13-14)

ถ้าได้ 400 Bad Request

สาเหตุยอดฮิต 3 อย่าง

(1) ลืมเปลี่ยน Body เป็น raw + JSON จึงส่งไปเป็นข้อความเปล่า (2) JSON พิมพ์ผิด เช่นมีลูกน้ำเกิน (3) ชื่อใน `model` ไม่ตรงกับ `key` จริงที่ได้จาก Lab 1

โหมด OpenAI-compatible

เป้าหมาย: ยิงแบบเดียวกับที่โลกทั้งใบใช้กับ OpenAI — ไบรารีสำเร็จรูปเกือบทั้งหมดใช้รูปแบบนี้ได้ทันที

Method: **POST** URL: `{{baseUrl}}/v1/chat/completions` (ไม่มีคำว่า `api` นำหน้า)

● JSON BODY

```
{
  "model": "ibm/granite-4-micro",
  "messages": [
    { "role": "system", "content": "ตอบเป็นภาษาไทยเสมอ" },
    { "role": "user", "content": "ยกตัวอย่าง HTTP status code ที่เจอบ่อย 3 ตัว" }
  ],
  "temperature": 0.7,
  "max_tokens": 300,
  "stream": false
}
```

ความต่างที่ต้องจำ

	Native <code>/api/v1/chat</code>	OpenAI-compatible <code>/v1/chat/completions</code>
ช่องใส่ข้อความ	<code>input</code> (ข้อความเดียว)	<code>messages</code> (อาร์เรย์ role/content)
คำสั่งระบบ	<code>system_prompt</code>	<code>message</code> ที่ <code>role: "system"</code>
จำกัดความยาว	<code>max_output_tokens</code>	<code>max_tokens</code>
ความจำ	จำได้ (stateful)	ไม่จำ — ส่งประวัติทั้งหมดเองทุกครั้ง
คำตอบอยู่ที่	<code>output[0].content</code>	<code>choices[0].message.content</code>

นี่คือหน้าตาของ stateless ตามหลัก REST — เซิร์ฟเวอร์ไม่จำอะไรเลย client ต้องแบกประวัติไปเองทั้งหมด

Endpoint อื่นในชุด OpenAI-compatible

Method + Path	หน้าที่
GET /v1/models	รายชื่อโมเดล (รูปแบบเดียวกับ OpenAI)
POST /v1/chat/completions	สนทนา รองรับข้อความและรูปภาพ
POST /v1/embeddings	แปลงข้อความเป็นเวกเตอร์ (ใช้ทำ RAG/ค้นหา)
POST /v1/completions	เติมข้อความแบบเก่า (legacy)
POST /v1/responses	รูปแบบ Responses API

● ตัวอย่าง EMBEDDINGS

```
{ "model": "text-embedding-nomic-embed-text-v1.5",
  "input": "แซตบอทสำหรับนักศึกษาไอที" }
```

คุยต่อเนื่องด้วย `previous_response_id`

- 1 ยัง `POST /api/v1/chat` สอบแรก: `"input": "ประเทศไทยมีกี่จังหวัด"`
- 2 คัดลอกค่า `response_id` จากคำตอบ (เช่น `resp_0182bd7c...`)
- 3 ยังรอบสอง โดยไม่ต้องเล่าเรื่องเดิมซ้ำ พร้อมแนบ `previous_response_id`

● รอบสอง

```
{
  "model": "ibm/granite-4-micro",
  "input": "แล้วภาคเหนือมีกี่จังหวัด",
  "previous_response_id": "resp_0182bd7c479d7451..."
}
```

ทำไมถึงทำแบบนี้ได้

เพราะ `store` มีค่าเริ่มต้นเป็น `true` ถ้าตั้ง `"store": false` จะไม่มี `response_id` กลับมา และคุยต่อแบบนี้ไม่ได้

สตรีมคำตอบแบบทยอยพิมพ์

เพิ่ม `"stream": true` ลงใน body

```
{ "model": "ibm/granite-4-micro",  
  "input": "เล่านิทานสั้น ๆ เกี่ยวกับโปรแกรมเมอร์",  
  "stream": true }
```

คำตอบจะไม่มาเป็นก้อนเดียว แต่ทยอยมาเป็นเหตุการณ์ย่อย ๆ ผ่านเทคนิคชื่อ **SSE (Server-Sent Events)** — นี่คือนวัตกรรมที่ทำให้เว็บเซตบออกแสดงตัวอักษรไหลออกมาทีละตัว

โหลดโมเดลเข้าหน่วยความจำ

POST

{{baseUrl}}/api/v1/models/load

```
{
  "model": "ibm/granite-4-micro",
  "context_length": 8192,
  "flash_attention": true,
  "offload_kv_cache_to_gpu": true
}
```

- `context_length` — ยิ่งใหญ่ยิ่งจำได้เยอะ แต่กินหน่วยความจำมากขึ้น
- `flash_attention` — เทคนิคเร่งความเร็ว
- `offload_kv_cache_to_gpu` — เก็บแคชไว้บน VRAM ของการ์ดจอ

ปลดโมเดล และดาวน์โหลดโมเดลใหม่

POST /api/v1/models/unload

```
{ "model": "ibm/granite-4-micro" }
```

POST /api/v1/models/download

```
{ "model": "ibm/granite-4-micro" }
```

ได้ `job_id` กลับมา แล้วเช็คด้วย GET /download/status/<job_id>

ลองยิง GET /api/v1/models ก่อนและหลัง load แล้วเทียบฟิลด์ loaded_instances

12 / 18

มี key กับ ไม่มี key ต่างกันอย่างไร

- › เปิดใช้ API key ใน LM Studio
- › ใส่ token ใน Postman + แบบฝึกทดสอบ

เทียบให้เห็นภาพ

	โหมดไม่มี key (ค่าเริ่มต้น)	โหมดมี key
สวิตช์ Require Authentication	ปิด	เปิด
ต้องส่ง header Authorization	ไม่ต้อง	ต้อง ทุก request
ใครยังเข้ามาได้	ใครก็ได้ที่เข้าถึงพอร์ตนี้ได้	เฉพาะคนที่ถือ token ที่ถูกต้อง
ถ้าลืมใส่ token	—	401 Unauthorized
เหมาะกับ	เรียนรู้นบนเครื่องตัวเอง	เปิดให้เครื่องอื่น/เพื่อนในวง LAN เรียกใช้

อุปมา

ไม่มี key = ประตูห้องแอบเปิดเอาไว้ ใครเดินผ่านก็เข้าได้ · มี key = ต้องแตะบัตรนักศึกษาก่อนเข้า และบัตรแต่ละใบกำหนดได้ว่าเข้าห้องไหนได้บ้าง

ขั้นตอนเปิดใช้ API key ใน LM Studio

- 1 ที่หน้า Developer กด **Server Settings**
- 2 เปิดสวิตช์ **Require Authentication**
- 3 กดปุ่ม **Manage Tokens** → Create Token
- 4 ตั้งชื่อ token (เช่น `postman-week5`) แล้วเลือกสิทธิ์ (permissions)
- 5 **คัดลอก token เก็บทันที** — ปิดหน้าต่างแล้วดูซ้ำไม่ได้อีก ต้องสร้างใหม่

ต้องใช้ LM Studio เวอร์ชัน 0.4.0 ขึ้นไป

ระบบ token เพิ่งมีตั้งแต่ 0.4.0 ถ้าเวอร์ชันเก่ากว่านี้จะไม่เห็นสวิตช์ดังกล่าว

ใส่ token ใน Postman — ทำได้ 2 วิธี

วิธี A (แนะนำ)

- 1 เปิด request ที่ต้องการ → แท็บ Authorization
- 2 Auth Type เลือก **Bearer Token**
- 3 ช่อง Token ใส่ `{{token}}`

วิธี B: พิมพ์ HEADER เอง

Key	Value
Authorization	Bearer lms-xxxx...

จุดพลาดที่เจอบ่อยมาก

ต้องมีคำว่า **Bearer** เวนวสค แล้วตามด้วย token — ใส่ token เปล่า ๆ จะได้ 401 กันที และระวังการคัดลอกมาแล้วติดช่องว่างหรือขึ้นบรรทัดใหม่ท้าย token

เทียบเท่ากับคำสั่ง curl

● GET พร้อม TOKEN

```
curl http://localhost:1234/api/v1/models \  
-H "Authorization: Bearer $LM_API_TOKEN"
```

● POST พร้อม TOKEN

```
curl http://localhost:1234/api/v1/chat \  
-H "Authorization: Bearer $LM_API_TOKEN" \  
-H "Content-Type: application/json" \  
-d '{"model":"ibm/granite-4-micro","input":"Write a short haiku about sunrise."}'
```

แบบฝึกทดสอบความเข้าใจ

- 1 เปิด Require Authentication แล้วยิง `GET /api/v1/models` โดยไม่ใส่ token → ควรได้ `401`
- 2 ใส่ token ผิด ๆ เช่น `Bearer abcdef` → ควรได้ `401` เหมือนกัน
- 3 ใส่ token ที่ถูก → ได้ `200` พร้อมรายชื่อโมเดล
- 4 ปิด Require Authentication แล้วยิงโดยยังใส่ token ค้างไว้ → ยังได้ `200` (เซิร์ฟเวอร์ไม่ตรวจ)

มารยาทของนักพัฒนา

อย่า commit token ลง Git และอย่าฝัง token ไว้ในโค้ดฝั่งหน้าเว็บ เพราะใครก็เปิดดูได้ — ควรเก็บใน environment variable หรือฝังเซิร์ฟเวอร์ของคุณเอง

13-14 / 18

MCP — Model Context Protocol

- ตัวละครและองค์ประกอบของ MCP
- เรียกใช้ MCP ผ่าน API ของ LM Studio จริง

MCP คืออะไร

MCP (Model Context Protocol) คือมาตรฐานเปิดสำหรับเชื่อม “แอปที่ใช้ AI” เข้ากับ “ระบบภายนอก” เช่น ไฟล์ในเครื่อง ฐานข้อมูล เครื่องมือค้นหา หรือ API ของบริการอื่น

อุปมาที่เรืบทงการใช้

MCP เป็นเหมือน **พอร์ต USB-C ของโลก AI** — แทนที่แต่ละแอปจะต้องเขียนตัวเชื่อมกับแต่ละบริการเอง ทุกฝ่ายทำตามมาตรฐานเดียว เสียบเข้ากันได้หมด

REST API กับ MCP ต่างกันตรงไหน

	REST API	MCP
ออกแบบมาเพื่อ	โปรแกรมทั่วไปคุยกับเซิร์ฟเวอร์	โมเดล AI ค้นพบและเรียกใช้เครื่องมือได้เอง
รูปแบบข้อความ	HTTP + JSON อีสระ	JSON-RPC 2.0 โครงสร้างตายตัว
รู้ว่ามีอะไรให้เรียก	ต้องอ่านเอกสารเอง	ถามเซิร์ฟเวอร์ได้เลยด้วย <code>tools/list</code>
สถานะ	ตามหลักคือ stateless	stateful มีขั้นตอน <code>initialize</code> ก่อน
ช่องทางส่ง	HTTP	stdio หรือ Streamable HTTP

ตัวละคร และสิ่งที่ MCP server เปิดให้ใช้

ตัวละครใน MCP

- **MCP Host** — แอป AI ที่คุมทุกอย่าง (LM Studio, Claude Desktop, VS Code)
- **MCP Client** — ตัวเชื่อมที่ host สร้างขึ้น 1 ตัวต่อ 1 เซิร์ฟเวอร์
- **MCP Server** — โปรแกรมที่เปิดเครื่องมือ/ข้อมูลให้ใช้

Primitive	เมธอด
Tools (เครื่องมือ)	<code>tools/list</code> , <code>tools/call</code>
Resources (ข้อมูล)	<code>resources/list</code> , <code>resources/read</code>
Prompts (แม่แบบ)	<code>prompts/list</code> , <code>prompts/get</code>

หน้าตาข้อความจริง (1): จับมือกันก่อน

● INITIALIZE REQUEST

```
{
  "jsonrpc": "2.0", "id": 1, "method": "initialize",
  "params": {
    "protocolVersion": "2025-06-18",
    "capabilities": { "elicitation": {} },
    "clientInfo": { "name": "example-client", "version": "1.0.0" }
  }
}
```

ทุกข้อความมี `"jsonrpc": "2.0"` และมี `id` ไว้จับคู่คำถาม-คำตอบ ส่วนข้อความที่**ไม่มี** `id` เรียกว่า **notification** ไม่ต้องตอบกลับ

หน้าตาข้อความจริง (2): ถามและสั่งใช้เครื่องมือ

ถามว่ามีเครื่องมืออะไรบ้าง

```
{ "jsonrpc": "2.0", "id": 2,  
  "method": "tools/list" }
```

สั่งใช้เครื่องมือ

```
{ "jsonrpc": "2.0", "id": 3,  
  "method": "tools/call",  
  "params": { "name": "weather_current",  
    "arguments": { "location": "San Francisco" } } }
```

คำตอบที่ได้

```
{ "jsonrpc": "2.0", "id": 3,  
  "result": { "content": [  
    { "type": "text",  
      "text": "Current weather in SF: 68°F" } ] } }
```

ต่อ MCP ผ่าน API ของ LM Studio

ข้อดี: ไม่ต้องเขียน JSON-RPC เอง — แค่ประกาศในไฟล์ integrations ของ `/api/v1/chat`

เตรียมก่อน: เปิดสวิตช์ **Allow per-request MCPs** และใช้โมเดลที่มี `trained_for_tool_use: true`

● EPHEMERAL MCP – ประกาศสด ๆ

```
{
  "model": "ibm/granite-4-micro",
  "input": "What is the top trending model on hugging face?",
  "integrations": [{
    "type": "ephemeral_mcp",
    "server_label": "huggingface",
    "server_url": "https://huggingface.co/mcp",
    "allowed_tools": ["model_search"]
  }],
  "context_length": 8000
}
```

อธิบายฟิลด์ของ ephemeral MCP

ฟิลด์	ความหมาย
type	"ephemeral_mcp" = เซิร์ฟเวอร์ชั่วคราว ใช้แล้วจบไปในรอบนี้
server_label	ชื่อเล่นที่เราตัวเอง ใช้อ้างอิงในคำตอบ
server_url	ที่อยู่ของ MCP server
allowed_tools	จำกัดว่าให้ใช้เครื่องมือชื่อไหนได้บ้าง — ควรใส่เสมอเพื่อความปลอดภัย
headers	(ไม่บังคับ) header เพิ่มเติมสำหรับยืนยันตัวตนกับ MCP server

คำตอบจะมีชิ้นส่วนแบบ `tool_call` เพิ่มเข้ามา

```
{
  "type": "tool_call",
  "tool": "model_search",
  "arguments": { "sort": "trendingScore", "limit": 1 },
  "output": "...",
  "provider_info": {
    "server_label": "huggingface",
    "type": "ephemeral_mcp"
  }
}
```

อ่านลำดับเหตุการณ์ในอาร์เรย์ `output` ได้เลยว่า โมเดลตัดสินใจเรียกเครื่องมือชื่ออะไร ส่งพารามิเตอร์อะไรไป ได้ผลลัพธ์อะไรกลับมา แล้วจึงสรุปเป็นข้อความให้ผู้ใช้

แบบที่ 2: plugin — ใช้ตัวที่ตั้งค่าไว้ใน mcp.json

```
{
  "model": "ibm/granite-4-micro",
  "input": "เปิดหน้าเว็บ example.com แล้วสรุปให้หน่อย",
  "integrations": [{ "type": "plugin", "id": "mcp/playwright" }]
}
```

สองเงื่อนไขของแบบ plugin

(1) ต้องเปิดสวิตช์ Allow calling servers from mcp.json ก่อน (2) การเรียกแบบนี้**ต้องแบบ API token เสมอ** เพราะ MCP server ประเภทนี้อาจสั่งรันคำสั่งบนเครื่องคุณได้

15 / 18

แก้ปัญหาก็เจอบ่อย

> ตารางอาการ → สาเหตุ → วิธีแก้ ที่ใช้อ้างอิงได้ทันที

อาการ สาเหตุ วิธีแก้ (1/2)

อาการ	สาเหตุที่น่าจะเป็น	วิธีแก้
<code>ECONNREFUSED</code>	เซิร์ฟเวอร์ไม่ได้เปิด หรือพอร์ตผิด	เช็ก Status = Running และเลขพอร์ต
Postman ว่ายัง localhost ไม่ได้	เบราว์เซอร์เข้าถึงเครื่องคุณไม่ได้	ใช้ Postman แบบติดตั้งลงเครื่อง
<code>404 Not Found</code>	พิมพ์ path ผิด	ระวัง <code>/v1/chat/completions</code> กับ <code>/api/v1/chat</code> สลับกัน
<code>400 Bad Request</code>	JSON ผิดไวยากรณ์ / ลืมปิดบั้งค์	Beautify ตรวจสอบ JSON, ตรวจสอบ Body = raw+JSON

อาการ สาเหตุ วิธีแก้ (2/2)

อาการ	สาเหตุที่น่าจะเป็น	วิธีแก้
401 Unauthorized	ลืมใส่ token / ใส่ผิด / ลืมคำว่า Bearer	ตั้ง Authorization = Bearer Token
403 Forbidden	token ถูก แต่สิทธิ์ไม่ครอบคลุม	Manage Tokens → Edit → เพิ่ม permission
รอนานมากครั้งแรก	JIT กำลังโหลดโมเดล	ปกติ — ครั้งถัดไปเร็วขึ้น
CORS error ใน console	ยังไม่เปิด Enable CORS	เปิดสวิตช์ Enable CORS
เพื่อยังจากเครื่องอื่นไม่ได้	ปิด Serve on Local Network / ไฟร์วอลล์	เปิดสวิตช์ + อนุญาตพอร์ต 1234

16-18 / 18

สรุปและแบบฝึกหัด

> แบบฝึกหัดท้ายบท → Cheat Sheet → คัพท์และแหล่งอ้างอิง

แบบฝึกหัดท้ายบท (1/2)

- 1 **สำรวจ** — ยิง `GET /api/v1/models` แล้วตอบว่าเครื่องคุณมีโมเดลที่ตัว ตัวไหนรองรับ vision และตัวไหนรองรับ tool use
- 2 **คุณสมบัติ** — ใช้ `system_prompt` ให้โมเดลตอบเป็น “อาจารย์สุภาพ” แล้วเทียบ `temperature` 0.1 กับ 1.0
- 3 **เทียบสองรูปแบบ** — ถามคำถามเดียวกันผ่าน `/api/v1/chat` และ `/v1/chat/completions` แล้วอธิบายความต่างของ JSON
- 4 **สนทนาต่อเนื่อง** — ทำบทสนทนา 3 รอบด้วย `previous_response_id` ให้รอบที่ 3 อ้างถึงรอบที่ 1

แบบฝึกหัดท้ายบท (2/2)

- 1 **ความปลอดภัย** — เปิด Require Authentication สร้าง token แล้วบันทึกภาพ 3 กรณี: ไม่ใส่ token (401), ใส่ผิด (401), ใส่ถูก (200)
- 2 **วัดประสิทธิภาพ** — ยิงคำถามเดียวกัน 5 ครั้ง จด `tokens_per_second` และ TTFT หาค่าเฉลี่ย
- 3 **ขั้นสูง** — ใช้ `integrations` ต่อ MCP server แล้วชี้ในผลลัพธ์ว่าขึ้นไหนคือ `tool_call`
- 4 **ส่งงาน** — Export Postman Collection เป็นไฟล์ .json ส่งพร้อมรายงาน

ຍິ່ງເຮົາ: ໄມ່ມີ key ກັບ ມີ key

● ໄມ່ມີ KEY

```
GET http://localhost:1234/api/v1/models

POST http://localhost:1234/api/v1/chat
Content-Type: application/json

{ "model": "ibm/granite-4-micro",
  "input": "ສັງຄັດ" }
```

● ມີ KEY

```
POST http://localhost:1234/api/v1/chat
Content-Type: application/json
Authorization: Bearer lms-xxxxxxxxxxxx

{ "model": "ibm/granite-4-micro",
  "input": "ສັງຄັດ" }
```

ลำดับการตั้งค่าใน Postman

1. เลือก Method → 2. ใส่ URL → 3. แท็บ Body → raw → JSON
- 4. (ถ้ามี key) แท็บ Authorization → Bearer Token
- 5. กด Send แล้วอ่าน status code ก่อนอ่าน body

ตารางสรุปเส้นทาง

อยากทำ	ไปที่
ดูโมเดลทั้งหมด	GET /api/v1/models
คุยกับโมเดล (แบบ LM Studio)	POST /api/v1/chat
คุยกับโมเดล (แบบ OpenAI)	POST /v1/chat/completions
ทำ embeddings	POST /v1/embeddings
โหลด / ปลดโมเดล	POST /api/v1/models/load · /unload
ดาวน์โหลดโมเดล	POST /api/v1/models/download
ใช้เครื่องมือภายนอก	POST /api/v1/chat + integrations

ศัพท์ไทย-อังกฤษ (1/2)

อังกฤษ	ไทย	คำอธิบายสั้น
API	ส่วนต่อประสานโปรแกรมประยุกต์	ช่องทางให้โปรแกรมคุยกับโปรแกรม
Endpoint	ปลายทาง / จุดเรียกใช้	URL หนึ่งเส้นที่ทำงานอย่างหนึ่ง
Request / Response	คำขอ / คำตอบ	ขาไปและขากลับของ HTTP
Header	ส่วนหัว	ข้อมูลกำกับ request/response
Payload / Body	เนื้อหา	ข้อมูลจริงที่ส่งไป
Status code	รหัสสถานะ	ตัวเลข 3 หลักบอกผลลัพธ์
Stateless	ไร้สถานะ	เซิร์ฟเวอร์ไม่จำ request ก่อนหน้า
Idempotent	ทำซ้ำได้ผลเดิม	ยิ่งที่ครั้งผลลัพธ์เท่ากัน

ศัพท์ไทย-อังกฤษ (2/2)

อังกฤษ	ไทย	คำอธิบายสั้น
Token / API key	บัตรผ่าน / กุญแจ	สายอักขระใช้พิสูจน์ตัวตน
Authentication	การพิสูจน์ตัวตน	“คุณเป็นใคร”
Authorization	การอนุญาตสิทธิ์	“คุณทำอะไรได้บ้าง”
CORS	การแบ่งปันทรัพยากรข้ามต้นทาง	กฎที่เบราว์เซอร์ใช้กันเว็บอื่นเรียก API
Streaming / SSE	การส่งแบบทยอย	ส่งคำตอบทีละชิ้นระหว่างสร้าง
Token (ของโมเดล)	หน่วยคำ	ชิ้นข้อความย่อยที่โมเดลประมวลผล (คนละความหมายกับ API token)
Context length	ความยาวบริบท	ปริมาณข้อความที่โมเดลมองย้อนหลังได้
Quantization	การลดความละเอียด	บีบโมเดลให้เล็กลงเพื่อรันบนเครื่องธรรมดา

แหล่งอ้างอิง

- RESTful API — restfulapi.net (หลักการ REST, ข้อจำกัด 6 ข้อ)
- Stanford CS193X Lecture 10 — HTTP & REST APIs (PDF)
- LM Studio — REST API v1 / Quickstart / POST /api/v1/chat
- LM Studio — Authentication (เปิด token, header Bearer, ต้องใช้ 0.4.0+)
- LM Studio — OpenAI Compatibility Endpoints / REST API v0
- LM Studio — Using MCP via API
- Model Context Protocol — Architecture overview / Introduction
- JSON-RPC 2.0 Specification (jsonrpc.org)

ขอบคุณครับ

พร้อมเปิด Postman คู่กับ LM Studio แล้ว

รายวิชา chatbot-it-kmitl · สัปดาห์ที่ 5 · ภาพหน้าจออ้างอิงจาก LM Studio 0.4.16